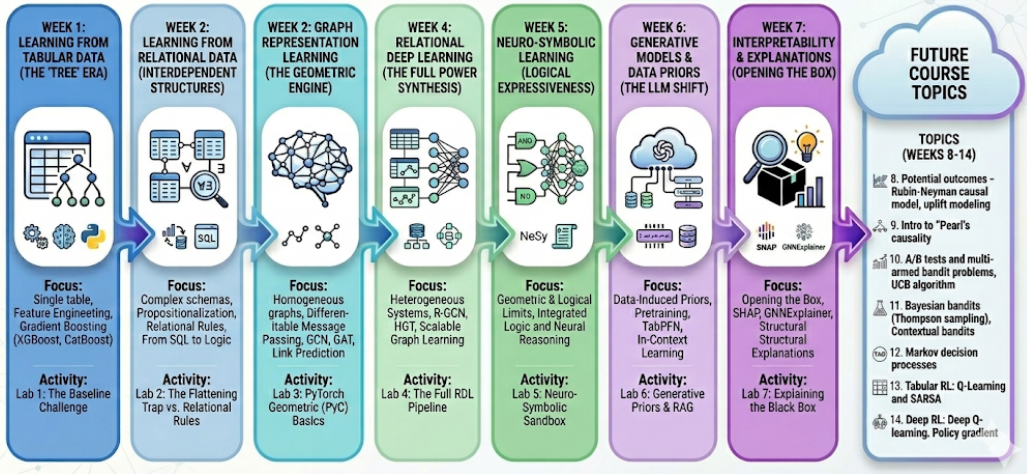


LEARNING FROM STRUCTURED DATA: A 7-WEEK JOURNEY

Narrative Arc: Increasing Structure → Increasing Inductive Bias → Increasing Reasoning Power



Course Map of Week 3: Graph Representation Learning

- 1 From Discrete Logic to Continuous Geometry
- 2 Shallow Embeddings & The Transductive Trap
- 3 The Message Passing Blueprint (MPNN Framework)
- 4 The GNN Design Space
- 5 Depth and Over-smoothing
- 6 End-to-End Training & PyTorch Geometric

The Limit of Week 2 & The “No Gradient” Wall

Recap: The Relational Era (Week 2)

- **Data:** Dynamic sets & Relational DBs.
- **Models:** Relational Decision Trees, ILP.
- **Mechanism:** Discrete logical splits.

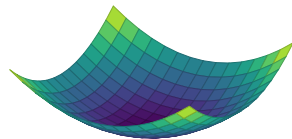
$$X \leftarrow \exists Y. \text{Tx}(X, Y) \wedge \text{Risk}(Y)$$



No Gradient $\nabla = 0$

The Differentiability Crisis

- Rules are boolean step functions.
- $\frac{\partial}{\partial x}(\text{SQL JOIN}) = \emptyset$ (No Gradients!).
- **Result:** Cannot use Gradient Descent.

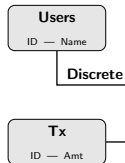


Smooth Continuous SGD
(Unreachable)

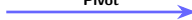
The Geometric Pivot

The Solution: Continuous Geometry

- Replace **Discrete Logic** with **Continuous Geometry**.
- Map relational topology to low-dimensional vector spaces.

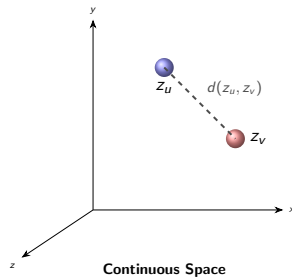


The Geometric
Pivot



The “Geometric Engine”

- **Nodes** → Dense, continuous vectors.
- **Edges** → Differentiable message pathways.
- **Optimization** → End-to-end backpropagation!



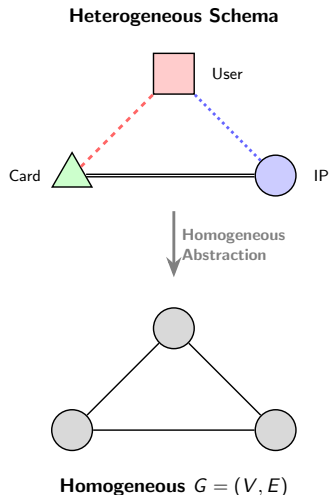
The Sandbox: The Homogeneous Abstraction

Reality: Heterogeneous Schemas

- Databases have multiple node/relation types.
- *Too complex to isolate core mathematics.*

Our Sandbox: The Homogeneous Graph

- Abstract the world to $G = (V, E)$.
- V : Set of identical Nodes.
- E : Set of identical Edges.
- **Goal:** Master *differentiable message passing*.



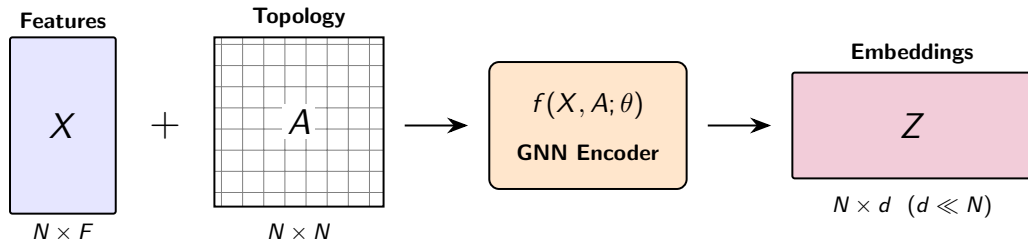
Formalizing Graph Representation Learning

The Input Data

- **Adjacency Matrix (A):**
 $A \in \{0, 1\}^{N \times N}$ (Topology).
- **Feature Matrix (X):** $X \in \mathbb{R}^{N \times F}$
(Node attributes).

The Optimization Goal

- Learn mapping function $f(X, A) = Z$.
- **Output (Z):** $Z \in \mathbb{R}^{N \times d}$ ($d \ll N$).
- Every node u receives dense vector $z_u \in \mathbb{R}^d$.



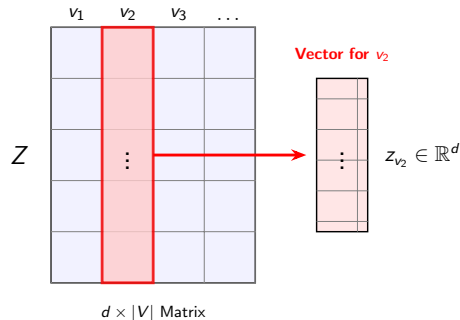
The Shallow Embedding Baseline

The “Shallow” Encoder

- Before GNNs, we used **Shallow Embeddings**.
- **Mechanism:** A simple lookup table.
- Every node $v \in V$ gets a unique, trainable vector.

Formalizing the Lookup

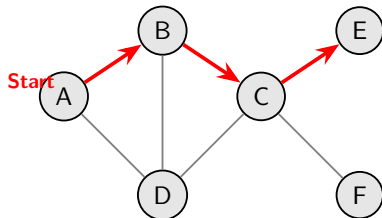
- **Embedding Matrix:** $Z \in \mathbb{R}^{d \times |V|}$
- **Encoder:** $\text{ENC}(v) = Z \cdot \mathbf{1}_v = z_v$
- *Goal:* Optimize Z so geometric distance $d(z_u, z_v)$ reflects graph similarity.



Random Walks as “Sentences” (DeepWalk)

Borrowing from NLP

- **Insight:** Can we use Word2Vec (Skip-Gram) on graphs?
- **Analogy:** Graph = Document, Nodes = Words.
- **Mechanism:** Generate “sentences” via random walks.

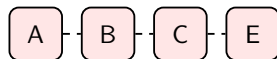


DeepWalk (2014)

- Start at node u , jump to random neighbors.
- Sequence: $(u, v_1, v_2, \dots, v_t)$.
- Optimize z_u to predict co-occurring nodes in the walk.

Nodes = Words
→

Sequential “Sentence”



Skip-Gram Optimization

Maximize Co-occurrence $P(v|z_u)$

Node2Vec: Controlling the Walk

The Limit of DeepWalk

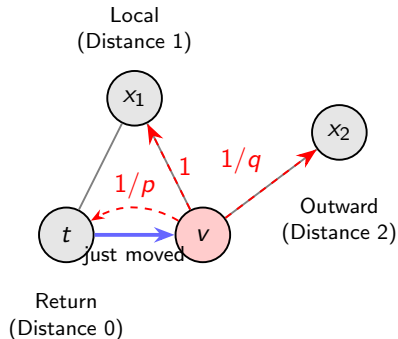
- Uniform random walks are rigid.
- Cannot capture distinct *types* of connectivity.

Biased Random Walks (Node2Vec)

- Introduces hyperparameters to control search:
- p (**Return**): Likelihood of backtracking.
- q (**In-out**): Likelihood of exploring outward.

The Exploration Tradeoff

- **Low** p : Breadth-First Search (BFS) $\rightarrow$ Local structural roles.
- **Low** q : Depth-First Search (DFS) $\rightarrow$ Global community/homophily.



The Optimization Objective

Maximizing Likelihood

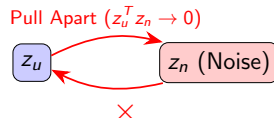
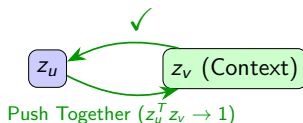
- Given node u , maximize probability of its neighborhood $N_R(u)$.

Cross-Entropy Loss

$$L = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|z_u)) \quad \text{where} \quad P(v|z_u) \propto \exp(z_u^T z_v)$$

The intuitive principle

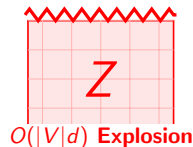
- Push z_u, z_v together for true pairs, pull apart for random noise pairs.



The Fatal Flaw: The Lookup Table Limit

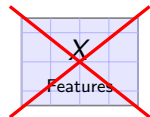
1. $O(|V|)$ Parameter Complexity

- No parameter sharing between nodes.
- Billion-node graph = Billion-row matrix. Does not scale.



2. Total Feature Blindness

- Optimizes purely on the adjacency structure (A).
- Completely ignores rich tabular node features (X)!



3. Inherently Transductive

- The model learns specific IDs, not generalizable.
- If a node wasn't in the training set $\implies$ no embedding.



Transductive:
Locked to
Training Graph

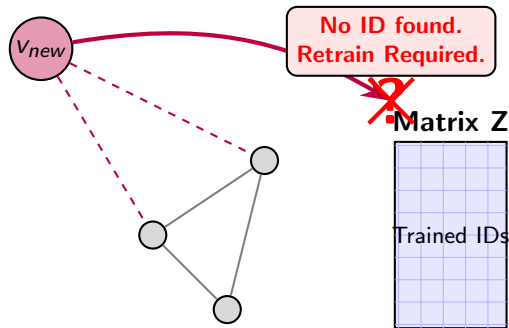
The Transductive Trap (The “Cold Start”)

The Industrial Reality

- Real-world graphs are dynamic (e.g., Fraud detection, Amazon).
- *Question:* What happens to our matrix when 10,000 new users join?

The “Cold Start” Failure

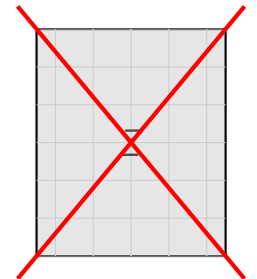
- New Node v_{new} arrives.
- No row exists for v_{new} in matrix Z !
- **Result:** Complete model failure. Must retrain the entire $O(|V|d)$ matrix from scratch.



The Inductive Imperative

Abandoning the Lookup Table

- We cannot learn static *vectors* of IDs.
- We must learn generalized **functions**.

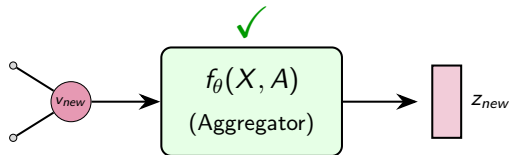


Transductive Table

The Transition to GNNs (Inductive)

- Learn a neural network $f_\theta(X, A)$ that aggregates neighborhoods.
- **Zero-Shot:** When v_{new} arrives, f_θ passes its features (X) to output z_{new} !

Inductive Function



The Universal Blueprint: MPNNs

Unifying Graph Learning

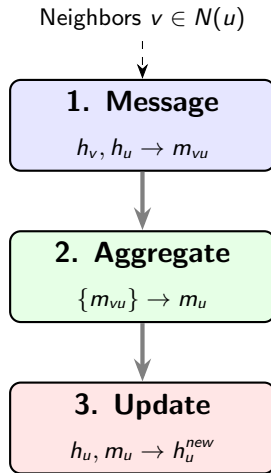
- We introduce a unifying GNN framework:
- *Message Passing Neural Networks (MPNNs)*.

The Core Paradigm

- A node's representation is dynamically determined...
- ...by iteratively aggregating its local topology.

The Three Phases

- 1. **Message**: Transform neighbor features.
- 2. **Aggregate**: Compress neighborhood to a vector.
- 3. **Update**: Evolve the target node's state.



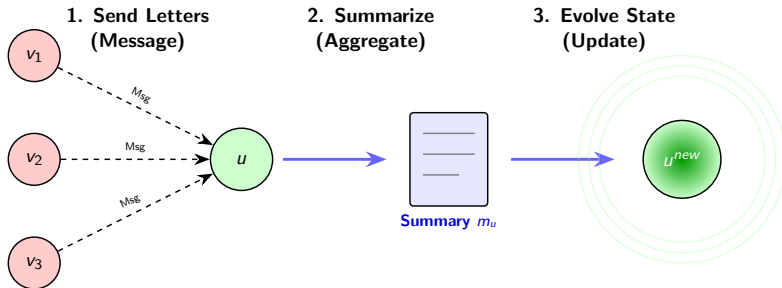
Intuition: The “Mailbox” Analogy

The Postal System

- Each node u has a personal mailbox.
- Edges are the delivery routes.

One Layer (Step k)

- Message:** Neighbors draft “letters” about their state.
- Aggregate:** Node u opens mailbox & summarizes them.
- Update:** and updates its knowledge with the summary.



Formalizing the MPNN Equation

The Universal Equation of Propagation

- At neural layer k , for a target node u :

$$h_u^{(k)} = \underbrace{\text{Update}^{(k)}}_{\text{3. Update}} \left(h_u^{(k-1)}, \underbrace{\text{Agg}^{(k)}}_{\text{2. Aggregate}} \left(\left\{ \underbrace{\text{Msg}^{(k)}(h_u^{(k-1)}, h_v^{(k-1)}, e_{uv})}_{\text{1. Message}} : v \in N(u) \right\} \right) \right)$$

Notation Breakdown

- $h_u^{(k)} \in \mathbb{R}^d$: Hidden state of node u at layer k .
- $N(u)$: The local topological neighborhood set of node u .
- e_{uv} : Optional edge features (e.g., bond type).

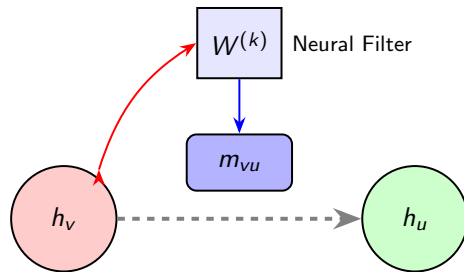
Phase 1: Message Generation

Creating the Data Packet

- $m_{vu}^{(k)} = \text{Msg}^{(k)}(h_u^{(k-1)}, h_v^{(k-1)}, e_{uv})$
- Transforms neighbor v 's state into a message for u .

Implementation Choices

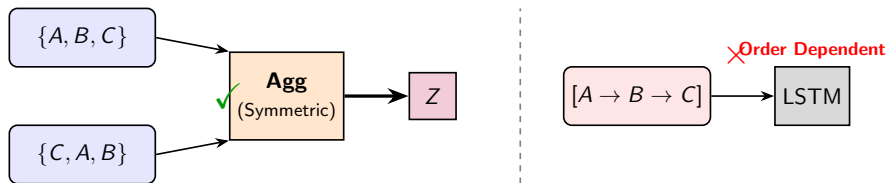
- **Simple Linear:** $m_{vu} = W^{(k)} h_v$
(Ignores receiver & edge).
- **Edge-Conditioned:** $m_{vu} = \text{MLP}(e_{uv}) \cdot h_v$
(Crucial e.g. for Chemistry).
- **Attention-based:** Learn interaction between h_u and h_v .



Phase 2: Permutation Invariant Aggregation

The Set Problem

- Neighborhoods are **Unordered Sets**, not Sequences!
- $\{v_1, v_2, v_3\} \equiv \{v_3, v_1, v_2\}$
- Standard RNNs/LSTMs fail here (they expect order).



Standard Aggregators ($\text{Agg}(A, B) = \text{Agg}(B, A)$)

- **Sum:** $\sum m_{vu}$ (Retains structural volume/degree).
- **Mean:** $\frac{1}{|N(u)|} \sum m_{vu}$ (Normalizes large hubs).
- **Max:** $\max(m_{vu})$ (Selects strongest feature signals).

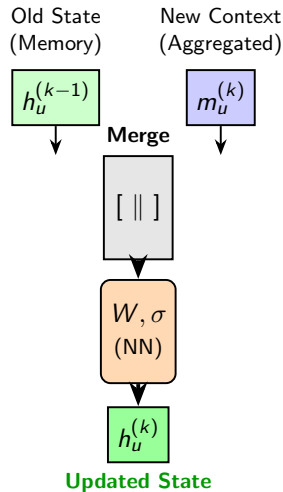
Phase 3: The Update Step

Evolving the Node State

- $h_u^{(k)} = \text{Update}^{(k)}(h_u^{(k-1)}, m_u^{(k)})$
- Merges the node's **past memory** with **new context**.

Implementation Choices

- **Concatenation (GraphSAGE):**
 $h_u^{(k)} = \sigma(W \cdot [h_u^{(k-1)} \parallel m_u^{(k)}])$
- **Summation (GCN):**
 $h_u^{(k)} = \sigma(W \cdot (h_u^{(k-1)} + m_u^{(k)}))$
- **Gated (GG-NN):**
 $h_u^{(k)} = \text{GRU}(h_u^{(k-1)}, m_u^{(k)})$



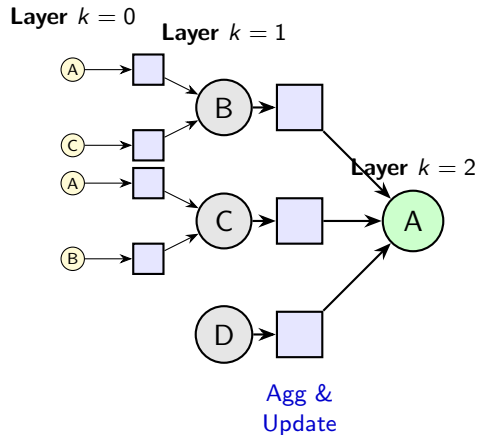
The Receptive Field: Unrolling the Graph

Stacking MPNN Layers (K)

- **Layer $k = 1$:** Node sees 1-hop neighbors.
- **Layer $k = 2$:** Node sees 2-hop neighbors (neighbors of neighbors).
- **Receptive Field:** K layers = K hops of topological context.

Tree-Structured Computation

- Each node dynamically defines its own neural network architecture!
- The computation graph unfolds as a rooted tree.



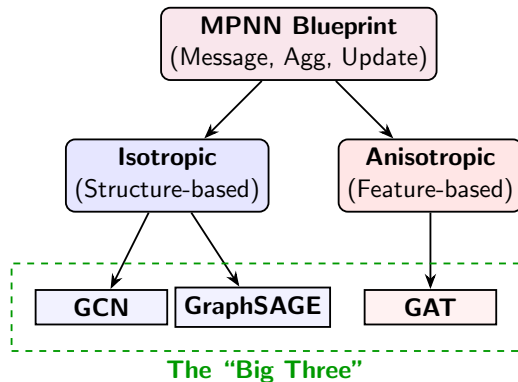
Operationalizing the MPNN Blueprint

The Design Space

- MPNN blueprint: **Message, Aggregate, Update.**
- How do we choose the exact functions?

Isotropic vs. Anisotropic Filters

- **Isotropic:** Treats all neighbors equally (or by structure).
- *Examples:* GCN, GraphSAGE.
- **Anisotropic:** Weights neighbors dynamically by features.
- *Example:* Graph Attention Networks.



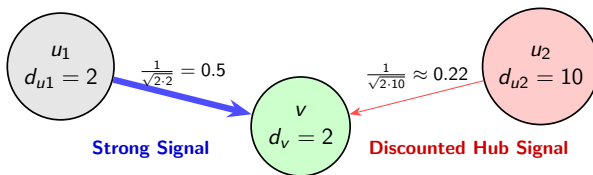
Graph Convolutional Networks (GCN)

GCN: The Isotropic Baseline (Kipf & Welling)

- **Concept:** “I am the normalized average of my neighbors.”
- Uses a simple linear message and a sum aggregation.

$$h_v^{(k)} = \sigma \left(\sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{d_v d_u}} W^{(k)} h_u^{(k-1)} \right)$$

- $N(v) \cup \{v\}$: Aggregates neighbors *and* itself (Self-loop).
- $W^{(k)}$: Learnable weight matrix.
- $\frac{1}{\sqrt{d_v d_u}}$: Symmetric degree normalization.



GCN: The Matrix Formulation

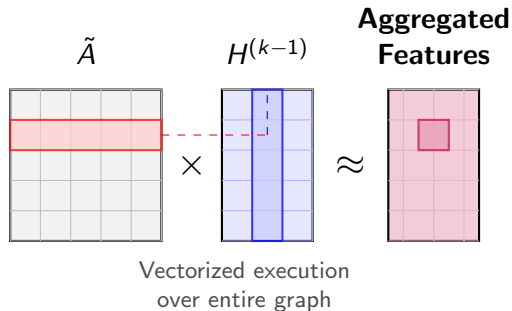
Scaling to the Full Graph

- Node-level sums are slow. We use Matrix Multiplication!
- **Adjacency with Self-Loops:**
 $\tilde{A} = A + I$
- **Degree Matrix:** $\tilde{D}$ (diagonal matrix of $\tilde{A}$ row sums)

The Differentiable Forward Pass

GCN Layer Operation

$$H^{(k)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(k-1)} W^{(k)} \right)$$



- $\tilde{A}H$: Sums up neighbor vectors.
- $\tilde{D}^{-1/2} \dots \tilde{D}^{-1/2}$: Applies normalization.
- Highly optimized for GPU parallelization.

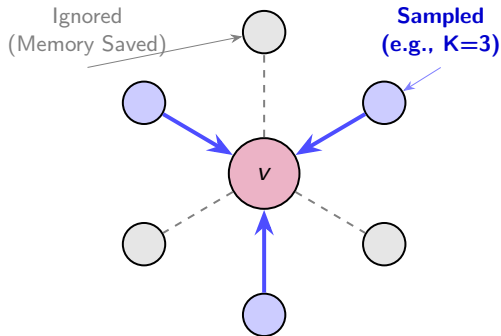
GraphSAGE: Concatenation & Sampling

The Limit of GCNs

- GCNs *sum* the target node with its neighbors.
- This dilutes the node's unique individual features!

GraphSAGE (SAmple and aggreGatE)

- **1. Separate Update (Concatenation):**
Keeps node's history distinct from its neighborhood context.
- **2. Neighborhood Sampling:** Do not aggregate *all* neighbors. Sample a fixed-size subset to ensure scalable memory footprint.



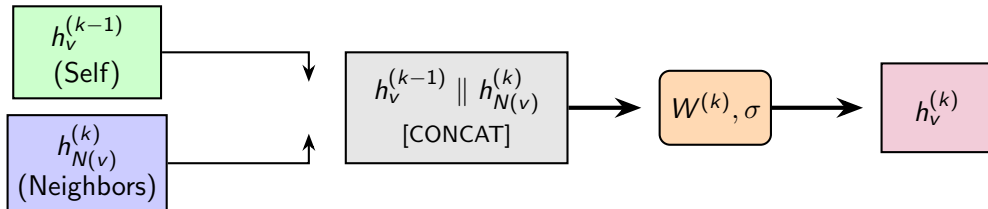
GraphSAGE: The Mathematics

Two-Stage Aggregation

- Unlike GCN, GraphSAGE strictly separates Aggregation from the Update.

The Formal Equations

- Stage 1: Aggregate Neighbors:** $h_{N(v)}^{(k)} \leftarrow \text{AGG} \left(\{h_u^{(k-1)}, \forall u \in N(v)\} \right)$
- Stage 2: Update via Concatenation:**
 $h_v^{(k)} \leftarrow \sigma \left(W^{(k)} \cdot \text{CONCAT}(h_v^{(k-1)}, h_{N(v)}^{(k)}) \right)$



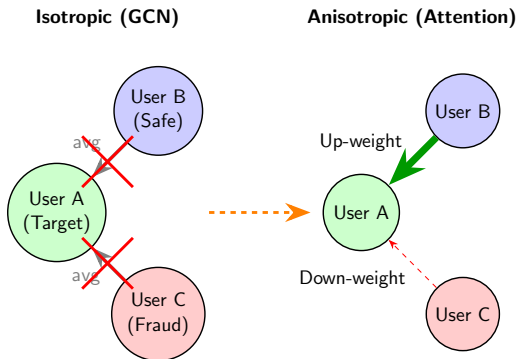
The Anisotropic Motivation

The Flaw of Isotropic Filters

- GCN/GraphSAGE sum/mean *all* neighbors.
- *Engagement Question*: What if your user is connected to a known fraudster?
- We do not want to average their features!

The Need for Anisotropy

- Not all edges are equally important to a specific task.
- We need a mechanism to dynamically **down-weight noisy edges** and **up-weight crucial edges**.
- **Solution**: Feature-based Attention.



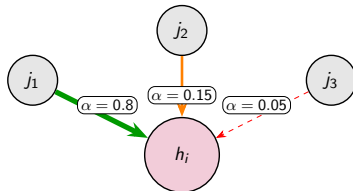
Graph Attention Networks (GAT)

GAT: Learning Who to Listen To

- **Concept:** “I assign an importance score to every neighbor.”
- Replaces static GCN normalization with dynamic **Attention Coefficients** (α_{ij}).

$$h_i^{(k)} = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} \mathbf{W} h_j^{(k-1)} \right)$$

- $\mathbf{W} h_j$: The projected message from neighbor j .
- α_{ij} : The learned attention weight (importance of j to i).
- If $\alpha_{ij} \approx 0$, the edge is effectively ignored!



Calculating Attention (α_{ij})

Step 1: Compute the raw score (e_{ij})

- Concatenate sender & receiver, project via learnable vector $\mathbf{a}$.

$$e_{ij} = \text{LeakyReLU} \left(\mathbf{a}^T [\mathbf{W}h_i \parallel \mathbf{W}h_j] \right)$$

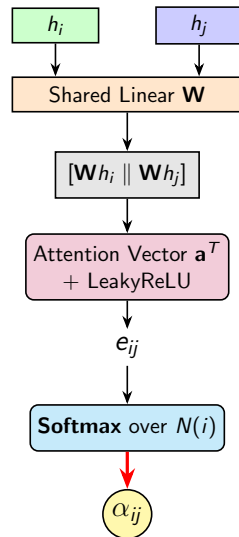
Step 2: Normalize over the neighborhood

- Use softmax to make weights sum to 1 across neighbors.

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})}$$

The Result:

- A localized probability distribution over edges.



Multi-Head Attention & The Transformer Link

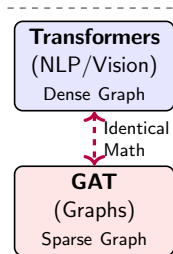
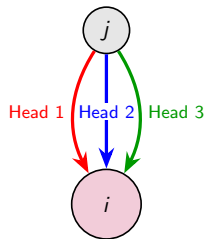
Stabilizing with Multi-Head Attention

- A single attention mechanism can be unstable.
- **Solution:** Use C independent attention heads, and concatenate the results!

$$h_i^{(k)} = \parallel_{c=1}^C \sigma \left(\sum_{j \in N(i)} \alpha_{ij}^c \mathbf{W}^c h_j^{(k-1)} \right)$$

GAT vs. Transformers (Week 6 Preview)

- A GAT is fundamentally a **Sparse Transformer**.
- *Transformer*: Dense attention (Fully connected sequence graph).
- *GAT*: Sparse attention (Restricted by Adjacency Matrix A).



The Temptation of Depth

The Deep Learning Dogma

- In Computer Vision (CNNs) & NLP (Transformers): Deeper is better.
- ResNet, GPT: 100+ layers of hierarchical feature extraction.

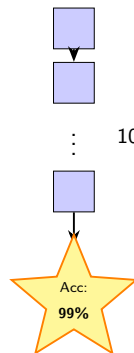
The GNN Reality

- Standard GNNs often peak at **2 to 3 layers**.
- Performance drastically *plummets* if stacked deeper!

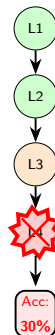
The Fundamental Question

- Why does adding layers destroy a Graph Neural Network?
- *Answer:* The topology of the Receptive Field.

Deep CNN



Deep GNN



The Exponential Receptive Field

Defining the Receptive Field

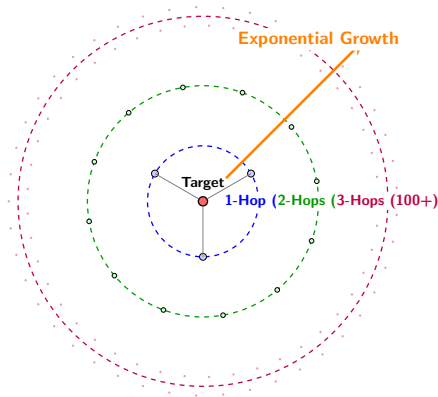
- **1 Layer:** Node aggregates its 1-hop neighbors.
- **K Layers:** Node aggregates a K -hop neighborhood.

The “Small World” Expansion

- Real-world graphs (Social, Web) are highly connected.
- The number of nodes in a K -hop radius grows *exponentially*.

The Macro-Level Collapse

- At 4 to 5 layers (hops), a node's receptive field encompasses almost the **entire graph**!



The Over-smoothing Phenomenon

Shared Fields = Shared Embeddings

- Consider two distant nodes, u and v .
- At $K = 4$ layers, their K -hop neighborhoods are nearly identical.

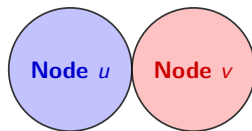
Repeated Aggregation is Averaging

- Both nodes aggregate the *exact same set* of global features.
- *Question:* If you mix all colors on a palette, what do you get?

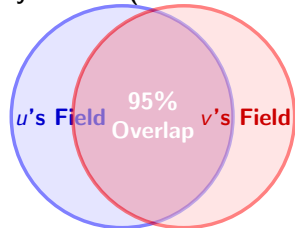
Over-smoothing

- All nodes converge to the **exact same embedding**.
- Individual node identities are completely erased!

Layer $K = 1$ (Distinct)



Layer $K = 4$ (**Over-smoothed**)



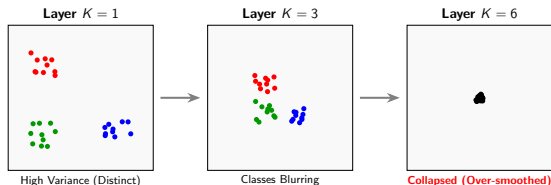
Mathematical Convergence (The Markov Chain)

GCN as a Random Walk

- Recall the GCN Matrix formulation: $H^{(K)} = (\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2})^K H^{(0)} W$
- Repeatedly multiplying by a normalized adjacency matrix is mathematically homologous to a Markov Chain random walk!

The Stationary Distribution

- As $K \rightarrow \infty$, the state converges to the graph's stationary distribution.
- $h_v^{(K)} \propto \sqrt{d_v}$ (Embedding just reflects node degree).
- Conclusion:** Deep message passing inherently destroys feature variance.



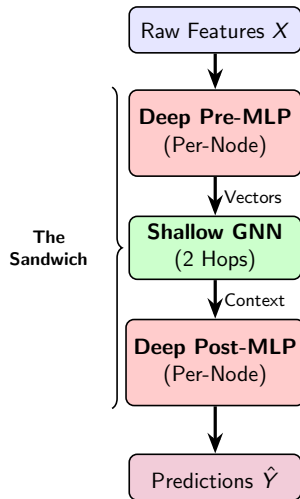
Solution 1: Pre- and Post-Process MLPs

Decoupling Expressivity from Depth

- We need deep neural networks for complex feature transformations.
- But we *cannot* use deep Message Passing.
- **Solution:** Use Multi-Layer Perceptrons on the nodes!

The “Sandwich” Architecture

- **Pre-process (Deep MLP):** Transform raw features (X) into dense vectors.
- **Message Passing (Shallow GNN):** 2-3 layers to mix topological context.
- **Post-process (Deep MLP):** Map the context vector to the final prediction.



Solution 2: Skip (Residual) Connections

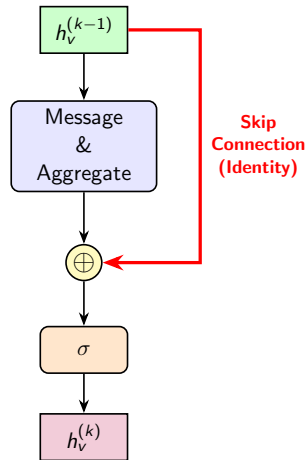
Preserving Node Identity

- If we *must* build a deeper GNN, how do we prevent the node's original identity from being washed away?
- Borrow from ResNet: **Skip Connections**.

Modified Update Function

$$h_v^{(k)} = \sigma \left(\mathbf{W} \cdot \text{AGG}(\dots) + \mathbf{h}_v^{(k-1)} \right)$$

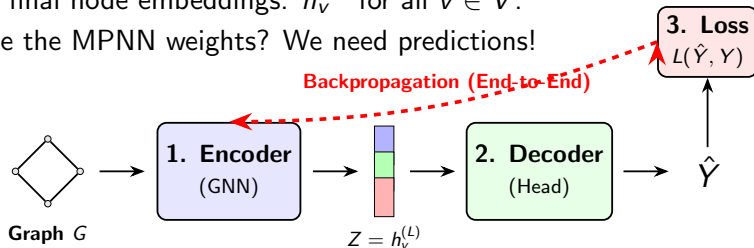
- Adds the node's previous state directly to the output.
- Creates a “gradient highway” during backpropagation.
- Safeguards the “Self” against neighborhood dilution.



The GNN Training Pipeline

The End-to-End Objective

- We have computed final node embeddings: $h_v^{(L)}$ for all $v \in V$.
- How do we optimize the MPNN weights? We need predictions!



The Pipeline

- **1. Encoder (GNN):** Maps Graph to Embeddings (Z).
- **2. Decoder (Head):** Maps Embeddings to Predictions ($\hat{Y}$).
- **3. Loss Function:** Computes error between $\hat{Y}$ and Labels (Y).

End-to-End Backpropagation

- Gradients flow from the Decoder, back through all MPNN layers!

Node-Level Decoders

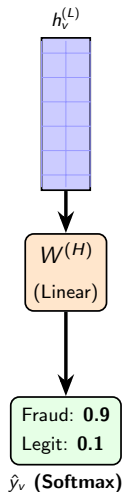
Predicting Node Properties

- **Task:** Classify individual nodes (e.g., Fraud vs. Legit).
- **Decoder:** Maps d -dimensional embedding to class logits.

Prediction Logic

$$\hat{y}_v = \text{Decoder}(h_v^{(L)}) = W^{(H)} h_v^{(L)}$$

- We apply a simple Linear layer (or MLP) to the final node vector.
- *Crucial:* The vector $h_v^{(L)}$ already contains L -hops of graph context!



Edge-Level Decoders (Link Prediction)

Predicting Relationships

- **Task:** Predict if an edge exists, or predict the edge's type.
- **Input:** Requires pairs of nodes: $\hat{y}_{uv} = \text{Decoder}(h_u^{(L)}, h_v^{(L)})$

Option 1: Concatenation + MLP

$$\hat{y}_{uv} = \text{MLP} \left([h_u^{(L)} \parallel h_v^{(L)}] \right)$$

- Highly expressive.
- Captures complex asymmetric interactions.

Option 2: Dot Product (Similarity)

$$\hat{y}_{uv} = (h_u^{(L)})^T h_v^{(L)}$$

- Fast, geometric.
- High dot product = High probability of a link!

Graph-Level Decoders & Virtual Nodes

Predicting the Macro-Structure

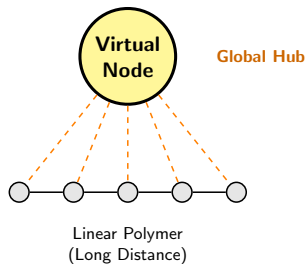
- **Task:** Classify the entire graph (e.g., Molecule Toxicity).
- **Decoder:** $\hat{y}_G = \text{Decoder}(\text{READOUT}(\{h_v^{(L)} \in G\}))$
- *READOUT*: Global Sum, Mean, or Max over all nodes.

The Long-Range Information Loss

- Global pooling struggles if graph is “long” (e.g., Polymers).
- Information gets diluted before reaching the other side.

The Fix: The “Virtual Super Node”

- Add a fake node connected to *every* real node in the graph!
- Reduces the maximum distance between all nodes to 2 hops.



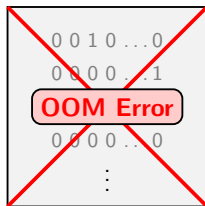
Enter PyTorch Geometric (PyG)

The Dense Adjacency Trap

- *Question:* A graph has 1 million nodes. How many entries in matrix A ?
- $10^6 \times 10^6 = 1$ Trillion (~ 4 TB of RAM!). **OOM Error**.

Sparse Representation: `edge_index`

- We cannot use dense matrices $A \in \mathbb{R}^{N \times N}$.
- **PyG Solution:** Sparse Coordinate Format (COO).
- `edge_index` $\in \mathbb{Z}^{2 \times |E|}$ (Stores only the existing edges).



PyG Source
→
Target

0	1	2	
1	2	0	

`edge_index` $\in \mathbb{Z}^{2 \times |E|}$
Only stores existing links

The MessagePassing Class

```
class MyGNN(MessagePassing):  
    def forward(self, x, edge_index):  
        return self.propagate(edge_index, x=x)  
  
    # 1. Generate Msg  
    def message(self, x_j):  
        return self.mlp(x_j)  
  
    # 2. aggregate() handled by aggr='add'  
  
    # 3. Evolve State  
    def update(self, aggr_out, x):  
        return aggr_out + x
```

Mapping Theory to Code

- PyG provides the MessagePassing base class.
- It automatically handles the complex sparse routing.

The Three Overridden Functions

- `propagate()`: The main trigger.
- `message()`: Defines the localized neural filter.
- `update()`: Merges memory and context.

Under the Hood: Gather and Scatter

How `propagate()` Works on GPUs

- Sparse operations are tricky for parallel hardware.
- PyG executes Message Passing via **Scatter/Gather** semantics.

The Two-Step Routing

- **1. Gather (Broadcast):** Node features X are duplicated and sent *to the edges* using the source IDs in `edge_index`.
- **2. Scatter (Reduce):** Edge messages are summed up and routed *back to the nodes* using the target IDs.

